PCSS (Percentage-Closer Soft Shadows) 정리 노트

원문: "Effect: Percentage Closer Soft Shadows" — Zgragselus, 2018-09-17
https://www.otte.cz/new/page/articles/post/8 (GameDev.net에도 "Effect: Area Light Shadows Part 1: PCSS"로 게재)
※ 원문 번역본이 아니라, 내용을 한국어로 요약·재구성한 학습용 노트입니다. 그림과 원문 코드는 원문을 참고하세요.

1. 글의 목적

저자는 자기 엔진에 **면광원(area light)**을 넣으면서, 면광원에 맞는 그림자, 즉 **본그림자(umbra)와 반그림자(penumbra)**가 제대로 생기는 그림자를 구현하려 했습니다. 글은 표준 섀도맵 → PCF → PCSS 순서로 발전 과정을 설명합니다.

2. 그림자 용어

영역	조건
본그림자 (umbra)	그 위치에서 광원의 어느 점도 보이지 않음
반그림자 (penumbra)	광원의 일부 점만 보임
그림자 없음	광원의 모든 점이 보임

대부분의 게임은 광원을 **크기가 없는 점**으로 단순화합니다. 그러면 가시성이 "보인다/안 보인다" 두 가지로 끝나고, 광원 시점에서 장면을 한 번 렌더링(섀도맵)하는 것으로 그림자 판정이 간단해집니다. 대신 현실의 반그림자는 표현할 수 없습니다.

3. 표준 섀도맵

광원 시점에서 그린 depth 맵을 한 번 읽고, 현재 지점의 광원 기준 깊이와 비교해 0 또는 1을 돌려줍니다. 결과는 경계가 딱딱하고 계단(aliasing)이 보이는 그림자입니다.

```
shadow = (섀도맵에서 읽은 depth < 현재 깊이) ? 0 : 1
```

4. PCF (Percentage-Closer Filtering)

주변 N×N 위치에서 여러 번 비교한 뒤 평균을 내서 경계를 부드럽게 만듭니다. 저자는 다음 조합을 썼습니다.

- 각 샘플 위치에서 `Gather`로 텍셀 2×2(4개)를 한 번에 가져옴
- 4개를 각각 깊이 비교해 0/1로 만든 뒤, **직접 bilinear 보간** (하드웨어 비교 샘플러 대신 수동 구현)
- 이걸 오프셋을 바꿔 가며 **5×5 위치**에서 반복하고 평균

구조 (개념 코드, 노트 작성자가 재구성)

```
t = frac(uv * 해상도 + 0.5)            // 텍셀 중심 기준 bilinear 가중치
for 오프셋 (i, j) in [-R..R] × [-R..R]:
    q = Gather(uv + (i, j) * 텍셀크기)   // 텍셀 2×2 depth
    r = (q >= 현재깊이) ? 1 : 0          // 4개 각각 비교
    합 += bilinear(r, t)                  // 비교 결과를 보간
shadow = 합 / (2R+1)²
```

이 대화에서 정리한 내용과 연결

- **비교 → 보간 순서**: depth를 섞지 않고, 비교 결과(0/1)를 보간한다는 PCF의 핵심을 그대로 따릅니다.
- **`+ 0.5`**: 텍셀 중심이 0.5 위치에 있으므로 가중치를 반 텍셀 보정한 것입니다. Luna 책 코드에는 빠져 있던 부분입니다.
- **크기 표기**: "5×5 PCF"는 샘플 위치 수 기준입니다. 각 위치가 텍셀 2×2를 보므로 텍셀 기준으로는 6×6입니다.
- **`SampleCmpLevelZero` 와의 관계**: `Gather` + 수동 비교 + 수동 bilinear 1세트가, 비교 샘플러로 `SampleCmpLevelZero`를 1번 호출하는 것과 같은 일을 합니다.

PCF의 한계

- 그림자 전체가 **똑같은 정도로** 흐려집니다. 물체와 닿는 곳은 선명하고 멀어질수록 흐려지는 현실의 반그림자를 표현하지 못합니다. 그래서 면광원에는 맞지 않습니다.
- 커널이 커질수록 샘플 수가 제곱으로 늘어 느립니다. VSM(Variance Shadow Maps)처럼 분리 가능한 필터를 쓰는 방법도 있지만, 빛 새는 현상 같은 다른 문제가 생깁니다.

5. PCSS (Percentage-Closer Soft Shadows)

점광원을 면광원으로 바꾸면 반그림자 폭이 다음 세 가지에 따라 달라집니다.

- 광원 크기
- 광원 ↔ 차폐물(blocker) 거리
- 광원 ↔ 그림자를 받는 면(receiver) 거리

PCSS는 이걸 흉내 내기 위해 **PCF 커널 크기를 픽셀마다 바꾸는** 기법입니다. 처리는 3단계입니다.

단계 1: 차폐물 탐색 (Blocker search)

- 현재 지점 주변의 일정 범위(search size)에서 섀도맵을 여러 번 읽습니다.
- 읽은 depth가 현재 깊이보다 **작은(광원에 더 가까운)** 샘플만 차폐물로 보고, 그 depth의 **평균**과 **개수**를 구합니다.
- 차폐물이 하나도 없으면 완전히 빛을 받는 곳이므로 바로 1을 반환하고 끝냅니다.
- 저자는 미리 정한 샘플 패턴을 픽셀마다 **무작위로 회전**시켜 샘플 수 대비 품질을 높였습니다.

> 이 단계는 depth 값 자체를 평균 냅니다. PCF에서 "depth를 평균 내면 안 된다"고 했던 것과 달리, 여기서는 그림자 판정이 아니라 **차폐물까지의 대략적인 거리**를 추정하는 용도라 평균을 써도 됩니다.

단계 2: 반그림자 크기 추정

차폐물과 받는 면이 평평하고 서로 평행하다고 가정하면, 닮은 삼각형으로 반그림자 폭을 구할 수 있습니다.

```
반그림자 폭 = 광원 크기 × (받는 면 깊이 − 평균 차폐물 깊이) / 평균 차폐물 깊이
```

- 차폐물이 받는 면에 **가까울수록**(분자가 작음) 반그림자가 좁아져 선명합니다.
- 차폐물에서 **멀어질수록** 반그림자가 넓어져 흐려집니다.
- 광원이 클수록 전체적으로 넓어집니다.

단계 3: 가변 크기 PCF

구한 반그림자 폭을 PCF 커널 크기로 써서 필터링합니다. 저자는 여기서도 회전시킨 샘플 패턴을 썼고, 샘플 배치를 바꾸면 광원 모양(원형, 사각형 등)도 흉내 낼 수 있다고 설명합니다.

전체 흐름 (개념 코드, 노트 작성자가 재구성)

```
(평균차폐물, 개수) = BlockerSearch(uv, 현재깊이, 탐색범위)
if 개수 == 0:
    return 1                                    // 차폐물 없음
반그림자 = 광원크기 * (현재깊이 - 평균차폐물) / 평균차폐물
return PCF(uv, 현재깊이, 커널크기 = 반그림자)
```

장단점

장점	단점
접촉부는 선명하고 멀어질수록 흐려지는, 그럴듯한 반그림자 소형~중형 면광원에 잘 맞음	노이즈 없이 하려면 샘플이 많이 필요해 비쌈 큰 커널에는 넓은 탐색 범위도 필요 대형 면광원에서는 문제가 생김

6. 원문 코드를 볼 때 참고할 점

- 차폐물 탐색 예제에서 누적하는 변수 이름과 나누는 변수 이름이 서로 달라 그대로는 컴파일되지 않습니다. 같은 변수로 맞춰야 합니다.
- 차폐물 개수가 0일 때도 평균을 내려고 나누기를 하므로, 실제 구현에서는 개수가 0인지 먼저 확인하는 게 안전합니다.
- PCSS 의사 코드의 "차폐물이 없는지" 검사는, 거리 값보다 **차폐물 개수**로 판단하는 편이 의도에 맞습니다.
- 예제의 비교는 `depth < 현재깊이 → 그림자`로, bias가 없습니다. 실제로는 이 대화에서 본 것처럼 bias가 필요하고, 커널이 커지는 PCSS에서는 큰 커널의 acne 문제(샘플별 깊이 보정)도 함께 고려해야 합니다.

7. 후속 내용 (원문에서 예고만 함)

- mipmap을 활용해 성능 부담을 줄이면서 더 큰 광원 크기를 허용하는 PCSS 변형
- 일종의 back-projection 기법

두 기법은 다음 글로 미뤄졌고, 첫 번째 기법의 시연 영상 링크만 원문에 있습니다.

한 줄 요약

> PCF는 커널 크기가 고정이라 그림자가 균일하게 흐려진다. PCSS는 **차폐물까지의 평균 거리**로 **반그림자 폭**을 계산하고, 그 폭만큼 **PCF 커널 크기를 픽셀마다 바꿔서** 접촉부는 선명하고 멀수록 흐린 면광원 그림자를 흉내 낸다.